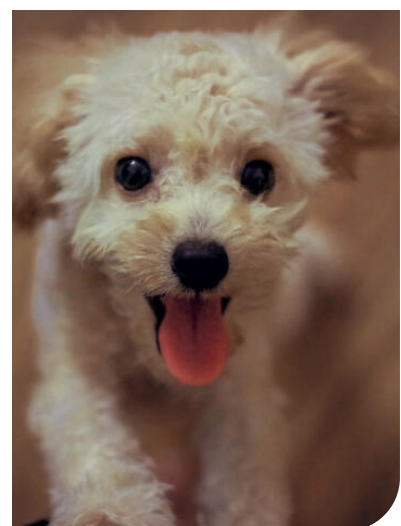
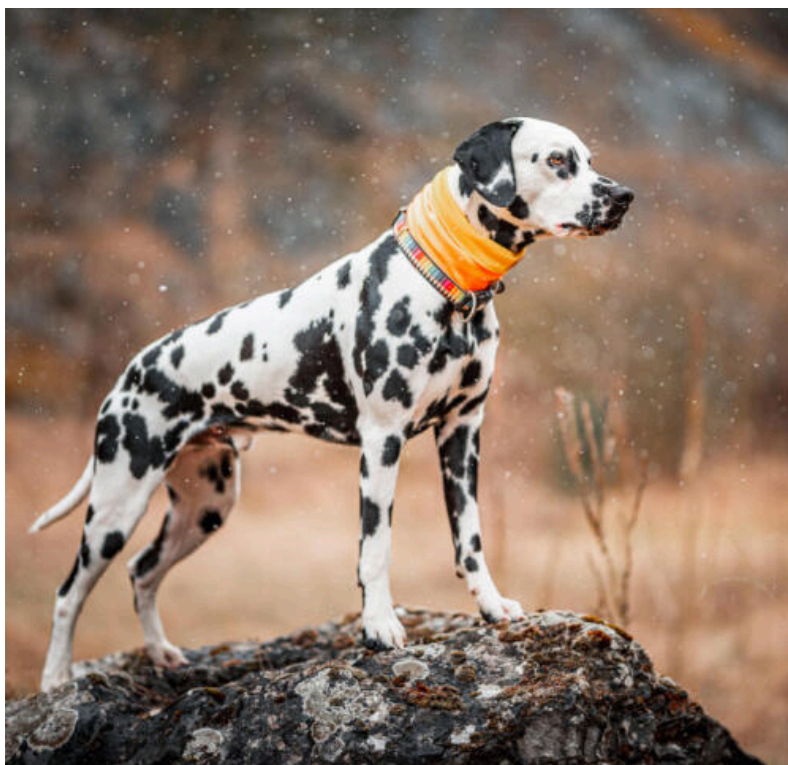
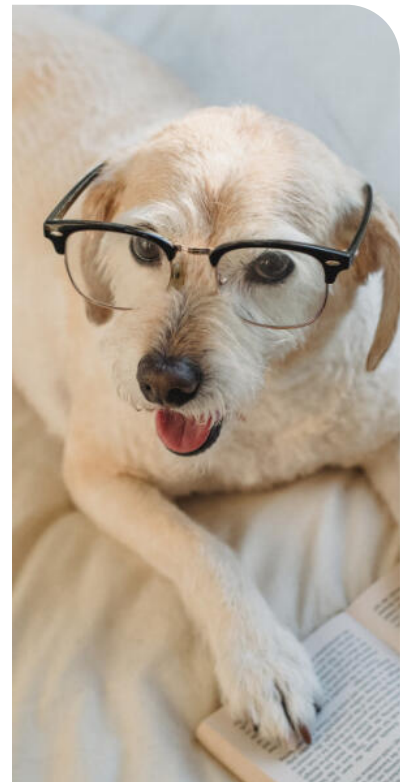


Figure 1: Erna places her images in neat rows

About Erna

Erna is a Typst library for aligning content (like images and graphics) in neat rows. It's easy to use. I made it for a friend who struggles with aligning images in LibreOffice. It's named after her. With Erna (the library), you can:

- Cram as many images as you see fit!
- Bravely mix sizes, resolutions and aspect ratios. Let the computer figure it out!
- Got some wiiiide images? Stack 'em up!
- Mix in text, shapes, and graphics too.

Erna promises to:

- ✓ fill all the available width;
- ✓ arrange the content in a rectangle;
- ✓ preserve all aspect ratios.

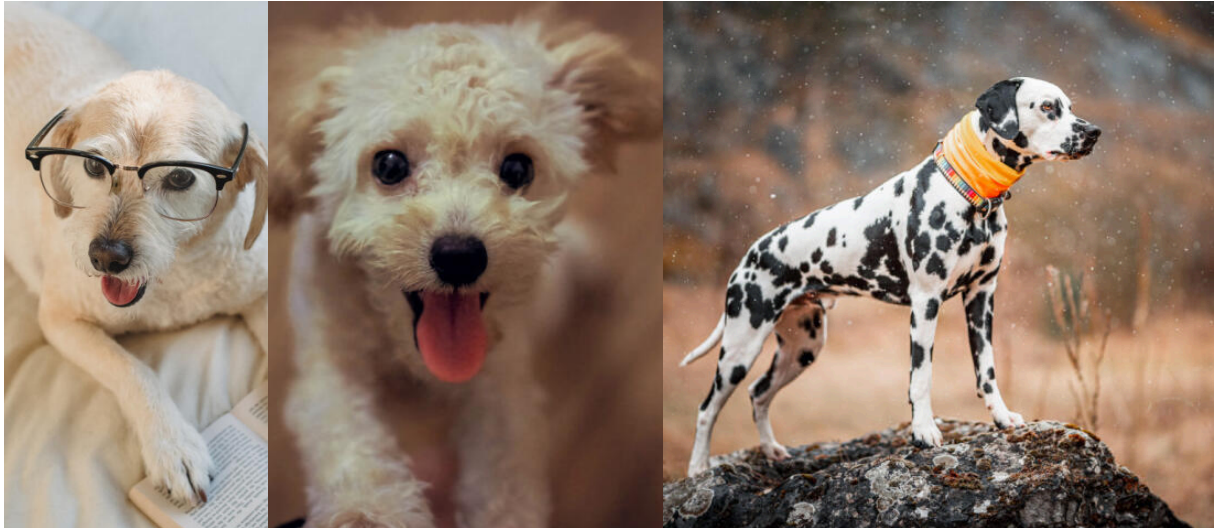


Erna, by tad-lispy.com

Basic use

At the heart of it is the `erna.row` function. It takes a list of content items (typically images), and scales them to equal height and to fit the available width, while preserving aspect ratio of each item. No cropping!

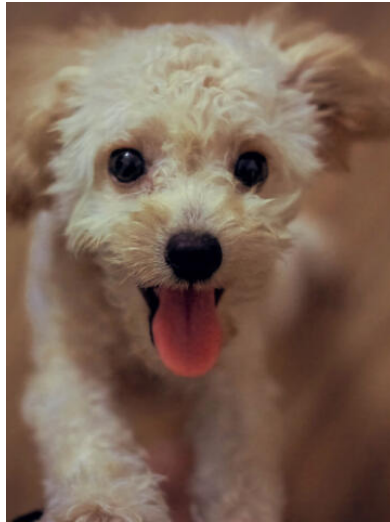
```
#erna.row(  
  image("300x600.jpg"),  
  image("360x480.jpg"),  
  image("500x480.jpg"),  
)
```



With gutters

You can put some space between the images with the gutter option.

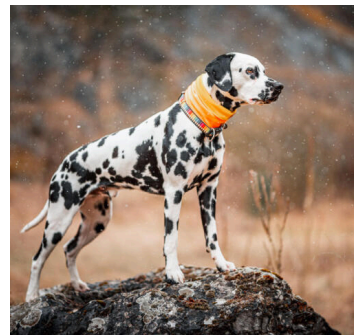
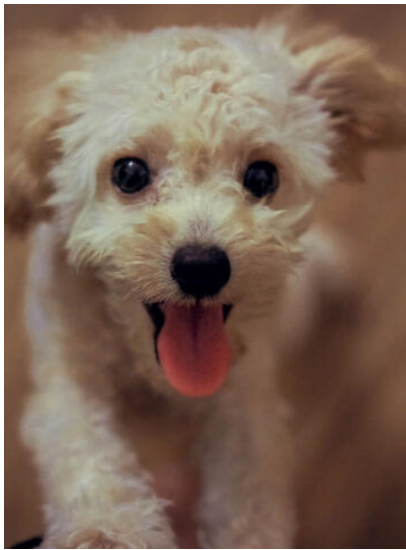
```
#erna.row(  
  gutter: 4pt,  
  image("300x600.jpg"),  
  image("360x480.jpg"),  
  image("500x480.jpg"),  
)
```



With stacks

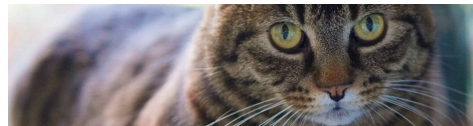
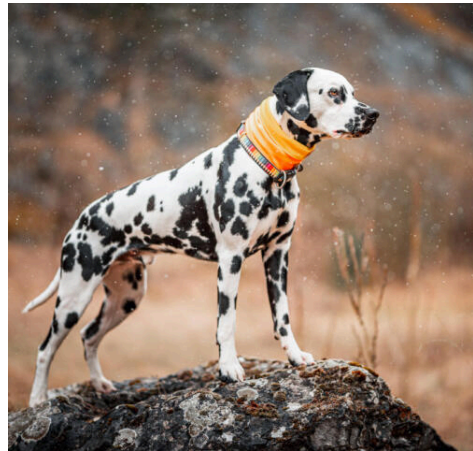
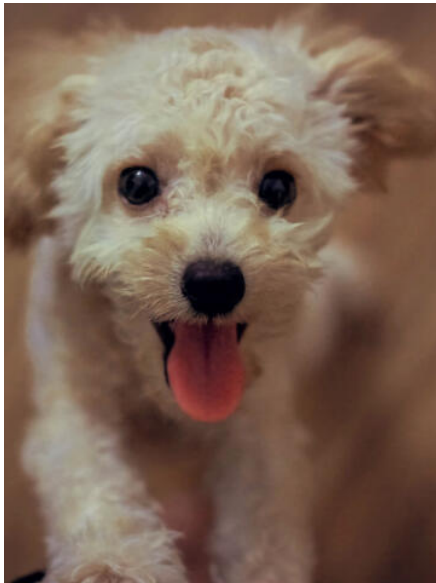
It's possible to place several images in a vertical stack. This is useful when mixing portraits and landscapes. Just group the paths in a parentheses to create a stack.

```
#erna.row(  
  gutter: 6pt,  
  
  // A stack of three  
  (  
    image("500x280.jpg"),  
    image("780x300.jpg"),  
    image("960x250.jpg")  
  ),  
  
  // A single image  
  image("360x480.jpg"),  
  
  // A stack of two  
  (  
    image("550x320.png"),  
    image("500x480.jpg"),  
  ),  
)
```



Multiple rows

```
#erna.multiple-rows(  
  gutter: 4pt,  
  // Row 1  
  (image("780x300.jpg")),  
  ( // Row 2  
    image("360x480.jpg"),  
    image("300x600.jpg"),  
    // A stack in a second row  
    ( image("500x480.jpg"), image("960x250.jpg" )),  
  )  
)
```



Other types of content

```
#figure(
  caption: [To prevent breaking Erna's content across multiple pages, wrap it in a
    _box_ or a _figure_.],
  erna.row(
    (
      rect("Cats"),
      image("420x800.jpg"),
      rect("dogs"),
    ),
    (
      image("960x250.jpg"),
      sym.amp,
      box(
        width: 180pt,
        height: 180pt,
        fill: blue,
        inset: 12pt,
        align(start + horizon)[
          #set text(fill: white)

          == Cats and Dogs

          Living together in _perfect_ harmony.

          Erna can mix and match different kinds of content. Not only
images, but also text, shapes, various containers... anything you like really!
        ]
      ),
    ),
    gutter: 12pt,
  )
)
```


Cats



dogs

&

Cats and Dogs

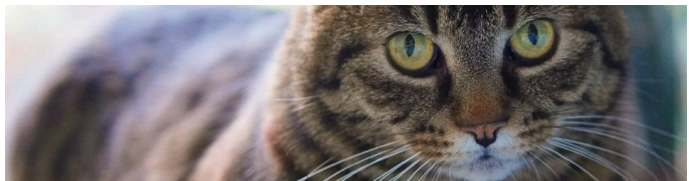
Living together in *perfect* harmony.

Erna can mix and match different kinds of content. Not only images, but also text, shapes, various containers... anything you like really!

Figure 2: To prevent breaking Erna's content across multiple pages, wrap it in a *box* or a *figure*.

Nested Erna

```
#box(erna.row(  
  (  
    image("420x800.jpg"),  
  ), (  
    image("960x250.jpg"),  
    {  
      show text: content => rect(  
        fill: yellow,  
        stroke: none,  
        align(center+horizon, content))  
      box(  
        width: 1000pt,  
        height: auto,  
        erna.row(  
          gutter: 60pt,  
          ("E", "r"),  
          ("r", "a"),  
        ),  
      )  
    },  
  ), gutter: 12pt,  
))
```



E

r

n

a

Kitchen Sink

A bit of a craziness below:

```
#let rects = (
  (width: 150pt, height: 150pt, fill: red.transparentize(40%)),
  (width: 75pt, height: 50pt, fill: green.transparentize(40%)),
  (width: 25pt, height: 40pt, fill: blue.transparentize(40%)),
).map(r => rect(..r, align(center + horizon, [
  #r.width × #r.height
])))

#let code = raw(lang: "rust", "
let x = resolve_axis(
  elem.x.get(styles),
  || size.as_ref().map(|size| size.x).map_err(Clone::clone),
  styles,
)?;
")

#box(erna.row(
  (
    // Green rectangle
    rects.at(1),

    // Dog's picture, fancy
    rotate(
      -4deg,
      box(
        image("420x800.jpg"),
        radius: 24pt,
        clip: true,
        stroke: 16pt + yellow
      )
    ),

    // Red rectangle
    rects.at(0),
  ), (
    // Raw text content
    [Hot#strong[dog]],

    // Blue rectangle
    rects.at(2),

    // Rust code
    code,
  ),
  gutter: 10pt
))
```

75pt × 50pt



150pt × 150pt

Hotdog

25pt

×

40pt

```
let x = resolve_axis(  
  elem.x.get(styles),  
  || size.as_ref().map(|size| size.x).map_err(Clone::clone),  
  styles,  
)?;
```